

TRUY VẤN DỮ LIỆU

Bài 4. Học xếp hạng

PGS.TS. Lê Thanh Hương
Trường Công nghệ thông tin và Truyền thông
Đại học Bách Khoa Hà Nội

Cách xếp hạng truyền thống

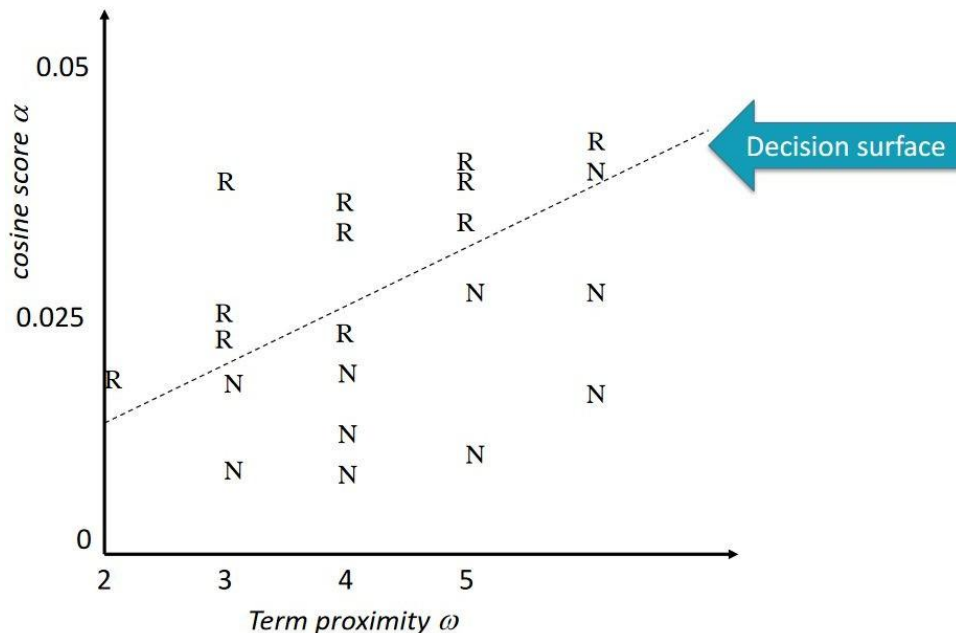
- Các hàm xếp hạng truyền thống trong IR sử dụng ít đặc trưng, ví dụ:
 - Term frequency
 - Inverse document frequency
 - Document length
- Có thể dễ dàng điều chỉnh trọng số bằng tay

Cách xếp hạng dùng học máy

- Các hệ thống hiện đại sử dụng nhiều đặc trưng:
 - Tần suất của từ truy vấn trong anchor text
 - Số liên kết trên trang
 - PageRank của trang
 - Lần cuối chỉnh sửa trang
 - Các đặc trưng khác tùy thuộc ứng dụng

Coi xếp hạng như bài toán phân loại

- Thu thập tập huấn luyện với các mẫu (q, d, r)
 - Độ liên quan r là giá trị nhị phân (thực tế có thể là đa lớp)



Learning to Rank

- Giả sử độ liên quan được sắp theo các mức

$$c_1 < c_2 < \dots < c_J$$

- Giả sử tập huấn luyện bao gồm các cặp tài liệu-truy vấn, ở dạng các vector đặc trưng ψ và thứ hạng liên quan c_i .

Learning to Rank

- **Input:** tập mẫu (là các đặc trưng của **query và documents**)

$$X = \{x_1, x_2, \dots, x_n\}$$

- **Output:** danh sách xếp hạng các mẫu dựa trên hàm score

$$Y' = \{x_{r1}, x_{r2}, \dots, x_{rn}\}$$

- **Mục tiêu:** tối ưu việc xếp hạng dựa trên các ma trận NDCG, MAP, MRR (đã học ở bài 2)

$$Y = \{x_{y1}, x_{y2}, \dots, x_{yn}\}$$

Nhắc lại

- MAP - Mean Average Precision

$$MAP = \frac{1}{|Q|} \cdot \sum \left(\frac{1}{R_q} \cdot \sum P@K_i \right)$$

- NDCG - Normalized Discounted Cumulative Gain.

$$DCG = rel_1 + rel_2/\log_2 2 + \dots rel_n/\log_2 n$$

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

- MRR - **M**ean **R**eciprocal **R**ank (Trung bình của giá trị nghịch đảo vị trí của kết quả phù hợp đầu tiên)

$$MRR(Q) = \frac{1}{|Q|} \cdot \sum_{q \in Q} \frac{1}{K_q}$$

Training & Test Data

Training

Relevance	Doc ID	Query ID	Query Length	Doc Length	Title SIM	Body SIM
1	1023	q1	2	103	0.50	0.45
2	27	q1	2	20	0.75	0.69
0	1023	q2	5	103	0.03	0.05
3	1532	q2	5	89	0.90	0.88
0	5	q3	1	1077	0.10	0.06

Not Seen in Training

Test

Relevance	Doc ID	Query ID	Query Length	Doc Length	Title SIM	Body SIM
?	104	q1	2	105	0.80	0.72

Predict



Phân loại mô hình học xếp hạng

- **Điểm đơn (Pointwise):** gán nhãn mức độ liên quan cho 1 tài liệu
 - Đầu vào: 1 tài liệu
 - Đầu ra: điểm hoặc nhãn mức độ liên quan
- **Cặp đôi (Pairwise):** Xác định thứ hạng của 2 tài liệu
 - Đầu vào: cặp tài liệu
 - Đầu ra: thứ tự ưu tiên trong cặp
- **Danh sách (Listwise):** xác định thứ tự cả tập tài liệu
 - Đầu vào: tập hợp tài liệu
 - Đầu ra: danh sách tài liệu được xếp hạng

Phân loại mô hình học xếp hạng

Given a query q and a set of documents $D = (d_1, \dots, d_n)$:

Pointwise

Input: Single candidate
 $x = (q, d_i)$



Loss: How accurate is the predicted score $s_i \approx y_i$?



Solution: Transform task into Regression.

Pairwise

Input: Pair of candidates
 $x_i = (q, d_i)$ and $x_j = (q, d_j)$



Loss: If $y_i > y_j$, then are the predicted scores $s_i > s_j$?



Solution: Transform task into Binary Classification.

Listwise

Input: Whole list of candidates
 $x_1 = (q, d_1) \dots x_n = (q, d_n)$



Loss: Takes into account position of all retrieved docs.



Solution: Incorporate evaluation metrics (e.g. DCG) into loss.

From [Francesco Casalegno](#) Post on towarddatascience



Phương pháp điểm đơn

Giảm thiểu bình phương hàm mất mát giữa điểm mức độ liên quan ước lượng và điểm mức độ liên quan thực tế.

perfect, excellent, good, fair, and bad

	Hồi qui	Phân loại	Hồi qui thứ bậc
Input	Vector đặc trưng: x		
Output	Số thực $y = f(x)$	Loại $y = \text{sign}(f(x))$	Loại theo thứ bậc $y = \text{thresh}(f(x))$
Model	Hàm xếp hạng $f(x)$		
Loss	Regression Loss	Classification Loss	Ordinal Regression Loss

Phương pháp điểm đơn

- **Sử dụng Linear Regression:**

$$f(x_i) = w^T x_i + b$$

w là vector trọng số (weights), b là bias.

- **Sử dụng Neural Network:**

$$f(x_i) = NN(x_i; \theta)$$

θ là tham số của mạng (các tầng dense, ReLU, dropout,...).

- **Sử dụng Decision Tree / GBDT / Random Forest:**

$$f(x_i) = \sum_{t=1}^T \alpha_t h_t(x_i)$$

mỗi h_t là một cây (weak learner), và α_t là trọng số của cây

Phương pháp điểm đơn

- Regression Loss: mục tiêu là làm $f(\mathbf{x}_i)$ gần với y_i nhất có thể.
- Hàm mất mát thường dùng là **Mean Squared Error (MSE)**:

$$L = \frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2$$

- Một biến thể khác:
 - **MAE (Mean Absolute Error)**:

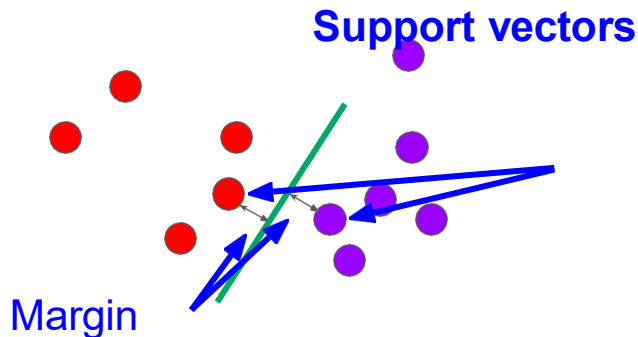
$$L = \frac{1}{N} \sum |f(x_i) - y_i|$$

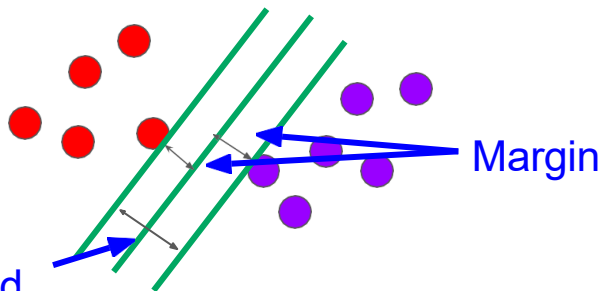
Phương pháp cặp đôi

- Với một cặp tài liệu, phương pháp này học nhằm đưa ra thứ tự tối ưu cho cặp đó và so sánh với thứ tự chuẩn.
- Mục tiêu:
 - Giảm số lần đảo ngược trong quá trình xếp hạng.
 - Giảm số trường hợp mà cặp kết quả bị xếp sai thứ tự so với thứ tự chuẩn.

SVMRank - Ranking SVM (Thorsten Joachims, 2002)

- Dựa trên Support Vector Machines (SVM), giải quyết các bài toán xếp hạng trong IR.
 - Xây dựng mô hình học xếp hạng dựa trên việc so sánh các cặp tài liệu và tối ưu hóa thứ hạng của chúng.
 - SVM là một mô hình tuyến tính dùng cho phân loại, tạo ra một đường thẳng hoặc siêu phẳng để phân chia dữ liệu thành các lớp.
 - **Support vectors**: đường nối các điểm dữ liệu gần nhất với siêu phẳng (các yếu tố quan trọng của tập dữ liệu).
- Margin**: Khoảng cách giữa siêu phẳng và điểm dữ liệu gần nhất từ mỗi tập.





SVM cố gắng tạo ra một biên sao cho khoảng cách giữa hai lớp là rộng nhất có thể.

- Giả sử $w = [w_1, w_2, \dots, w_d]$ là vector trọng số thì độ dài của vector w trong không gian d chiều

$$\|w\| = \sqrt{w_1^2 + w_2^2 + \dots + w_d^2}$$

$$\text{margin} = \frac{1}{\|w\|}$$

- SVMRank xếp hạng tài liệu dựa trên các cặp tài liệu và tối ưu hóa thứ tự của chúng thông qua kỹ thuật phân loại SVM
- **Đầu vào:**
 - Truy vấn q
 - Các tài liệu liên quan $d_1, d_2, \dots, d_n$ với nhãn (label) cho biết mức độ liên quan của từng tài liệu.
- **Mục tiêu:** xếp hạng các tài liệu sao cho tài liệu có nhãn cao hơn (liên quan hơn) sẽ đứng trước các tài liệu có nhãn thấp hơn.
- **Tạo ra các cặp tài liệu:**
 - Từ danh sách các tài liệu được gán nhãn, ta tạo ra các cặp tài liệu.

Xây dựng mô hình:

- SVMRank học 1 hàm tuyến tính dựa trên các cặp tài liệu (d_i, d_j) , với mục tiêu tối ưu hóa một hyperplane nhằm phân tách tốt nhất các (d_i, d_j) , nghĩa là tài liệu có liên quan hơn sẽ nằm ở phía đúng (+) của siêu phẳng so với tài liệu kém liên quan hơn.
- Ràng buộc với hàm mục tiêu

$$w \cdot (x_i - x_j) \geq 1 - \xi_{ij}, \forall (d_i, d_j)$$

- x_i và x_j là vector đặc trưng của tài liệu d_i và d_j .
- w là vector trọng số được học bởi mô hình.
- ξ_{ij} là biến sai số cho phép.

- **Mục tiêu:** tối thiểu hóa hàm mất mát dựa trên lỗi xếp hạng và chuẩn của vector trọng số w :

$$\min \frac{1}{2} ||w||^2 + C \sum \xi_{ij}$$

- C là tham số điều chỉnh sự đánh đổi giữa độ phức tạp của mô hình và lỗi huấn luyện.
- **Xếp hạng tài liệu:**
 - Sau khi huấn luyện, điểm cho các tài liệu được tính thông qua hàm $f(x)=w \cdot x$ với x là vector đặc trưng của tài liệu.

Ý nghĩa $\frac{1}{2} \| w \|^2$ trong hàm loss:

- Đây là thành phần regularization (chuẩn hóa mô hình) nhằm giới hạn độ lớn của $w \rightarrow$ tránh mô hình quá phức tạp hoặc overfit.
- Tối thiểu hóa $\| w \|^2$ tương đương với tối đa hóa khoảng cách (margin) giữa các cặp xếp hạng đúng và sai:
- Nếu chỉ tối ưu sai số $\sum \xi_{ij}$ mà không có $\| w \|^2$, mô hình có thể phóng đại trọng số w vô hạn để đạt đúng toàn bộ thứ tự — điều này dẫn tới overfitting và thiếu ổn định.

Ưu điểm:

1. **Hiệu quả trong việc xếp hạng:** SVMRank có thể đưa ra thứ hạng chính xác bằng cách tối ưu hóa thứ tự giữa các cặp tài liệu
2. **Áp dụng dễ dàng với các mô hình IR**

Nhược điểm:

1. **Tốn tài nguyên tính toán**
2. **Khả năng mở rộng:** SVMRank có thể gặp khó khăn khi xử lý các tập dữ liệu lớn do độ phức tạp tính toán cao khi làm việc với nhiều cặp tài liệu.

Nhận xét phương pháp cặp đôi

- **Ưu điểm**

- Tiến gần hơn một bước đến việc xếp hạng tài liệu so với việc dự đoán nhãn phân loại.

- **Nhược điểm**

- Một số truy vấn có nhiều cặp truy vấn hơn so với các truy vấn khác.
- Vẫn chưa tối ưu cho các độ đo trong IR
- Không nhận biết thứ hạng — ($d_1 > d_2$) không thể hiện rõ thứ hạng nào đang được so sánh: Hạng 1 với Hạng 2 hay Hạng 99 với Hạng 1000.

Phương pháp danh sách

- Các mô hình học được thiết kế để tối ưu chất lượng toàn bộ danh sách kết quả xếp hạng, thay vì chỉ tối ưu cho từng cặp tài liệu (pairwise) hoặc từng tài liệu riêng lẻ (pointwise).
- Tập trung vào việc tối ưu các chỉ số xếp hạng như NDCG, MAP, MRR
- Các phương pháp tiêu biểu: **AdaRank**, **LambdaRank**, **LambdaMART**

- AdaRank kết hợp nhiều mô hình xếp hạng yếu (weak rankers) h_t để tạo thành một mô hình mạnh hơn H_T .
- **Weak ranker** h_t có **trọng số** α_t - **độ tin cậy** của h_t
- Mục tiêu: tối ưu trực tiếp **chất lượng xếp hạng toàn bộ danh sách**, ví dụ theo **NDCG**, **MAP**, ...
- Lặp việc huấn luyện từng weak ranker: qì mà mô hình hiện tại không xếp hạng tốt được gán trọng số cao hơn. Sau mỗi lần học, mô hình sẽ cập nhật lại α_t → cải thiện độ chính xác trên các truy vấn khó.
- Mô hình tổng hợp sau T lần chạy:

$$H_T(x) = \sum_{t=1}^T \alpha_t \cdot h_t(x)$$

- → Ranker nào hiệu quả hơn sẽ có α_t lớn hơn.

- Khởi tạo tất cả các truy vấn được gán trọng số bằng nhau:

$$p_1(q_i) = \frac{1}{N}, \forall i = 1, 2, \dots, N$$

- Sau mỗi vòng lặp, độ cải thiện chất lượng xếp hạng (ví dụ NDCG) của mô hình hiện tại đối với mỗi truy vấn q_i là

$$Eval(H_{t-1} \oplus h_t, q_i)$$

- $H_{t-1}(x)$: mô hình mạnh tạm thời
- $h_t(x)$: weak ranker mới
- Độ lỗi của mô hình trên query q_i
$$E(q_i) = 1 - Eval(H_{t-1} \oplus h_t, q_i)$$

Các bước thực hiện:

1. Khởi tạo trọng số query: $P_1(q_i) = \frac{1}{N}$
2. Ở vòng lặp t : chọn weak ranker h_t tối ưu metric tổng, có trọng số $P_t(q_i)$
3. Tính α_t (độ mạnh của ranker)

4. Cập nhật trọng số query $P_{t+1}(q_i)$ để tăng trọng số các query mà h_t làm tệ

$$P_{t+1}(q_i) = \frac{P_t(q_i) \cdot \exp(-\alpha_t \cdot \text{Eval}(h_t, q_i))}{Z_t}$$

- Z_t : hệ số chuẩn hóa để các trọng số cộng lại = 1.
- α_t càng lớn $\rightarrow$ ảnh hưởng của ranker càng mạnh trong việc **điều chỉnh trọng số query** cho vòng sau.

5. Mô hình tổng hợp: $H_T(x) = \sum_{t=1}^T \alpha_t \cdot h_t(x)$

6. Dừng khi đạt tiêu chí hội tụ hoặc hết vòng lặp

AdaRank – Cập nhật α_t

- Ở mỗi vòng lặp t , chọn h_t và α_t để:

$$(\alpha_t, h_t) = \arg \max_{\alpha, h} \sum_i P_t(q_i) \cdot \text{Eval}(H_{t-1} \oplus (\alpha \cdot h), q_i)$$

- $\text{Eval}(h_t, q_i)$: đánh giá h_t cho query q_i
- $P_t(q_i)$: trọng số của query q_i ở vòng hiện tại

$$\alpha_t = \frac{1}{2} \ln \frac{1 + \text{Corr}_t}{1 - \text{Corr}_t} \quad \text{với} \quad \text{Corr}_t = \sum_i P_t(q_i) \cdot \text{Eval}(h_t, q_i)$$

- Corr_t : mức tương quan (correlation) giữa h_t và metric tổng thể
- Khi h_t xếp hạng tốt trên hầu hết các query có trọng số cao $\Rightarrow \text{Corr}_t$ cao $\Rightarrow \alpha_t$ lớn

- **Ưu điểm:**

- Tối ưu trực tiếp các chỉ số xếp hạng.
- Có thể đạt hiệu quả cao khi kết hợp các mô hình yếu thành một mô hình mạnh.

- **Nhược điểm:**

- Dễ overfitting nếu không có cơ chế dừng hợp lý.
- Không có khả năng mở rộng tốt cho dữ liệu lớn do cần huấn luyện nhiều mô hình con.

Đánh giá cách tiếp cận danh sách

- **Ưu điểm**

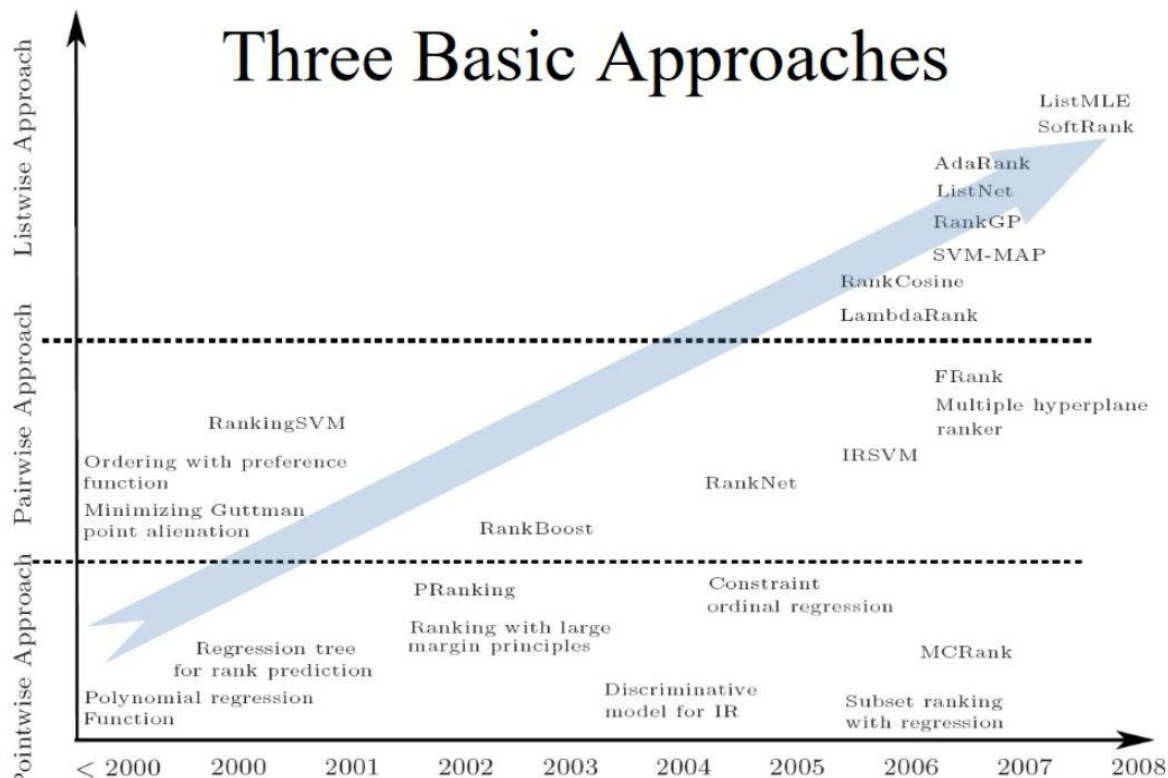
- Sử dụng tất cả các tài liệu liên quan đến một truy vấn làm ví dụ học
- Hàm loss có xét đến vị trí xếp hạng

- **Nhược điểm:**

- Cần nhiều dữ liệu huấn luyện

So sánh các cách tiếp cận

- Listwise > pairwise > pointwise ranking



Học không giám sát

- **CombSUM:** Kết hợp kết quả xếp hạng từ nhiều hệ thống tìm kiếm hoặc nhiều truy vấn khác nhau.
- Thuật toán này là một trong các phương pháp hợp nhất điểm số nhằm cải thiện chất lượng và độ chính xác của kết quả truy hồi thông tin.
- Giả sử có N hệ thống IR khác nhau trả về điểm số cho tài liệu d . Điểm số của tài liệu d từ hệ thống thứ i là $score_i(d)$.

$$Score_{\text{CombSUM}}(d) = \sum_{i=1}^N score_i(d)$$

- Tài liệu có điểm số cao hơn sẽ đứng đầu danh sách kết quả.

Giả sử có 3 hệ thống truy hồi thông tin khác nhau. Mỗi hệ thống trả về điểm số cho tài liệu d:

- Hệ thống 1: $\text{score}_1(d)=0.7$
- Hệ thống 2: $\text{score}_2(d)=0.8$
- Hệ thống 3: $\text{score}_3(d)=0.9$

Tổng điểm của tài liệu d:

$$\text{Score}_{\text{CombSUM}(d)} = 0.7 + 0.8 + 0.9 = 2.4$$

Đặc điểm của CombSUM

Ưu điểm:

- Đơn giản: chỉ cần cộng các điểm số từ nhiều hệ thống.
- Tính tổng quát: không cần điều chỉnh các hệ thống IR ban đầu, mà chỉ cần lấy tổng điểm số
- Hiệu quả trong nhiều trường hợp: CombSUM có thể mang lại kết quả tốt hơn khi các hệ thống IR khác phương pháp hoặc khác bộ dữ liệu.

Hạn chế:

- Không xem xét độ tin cậy của từng hệ thống
- Không chuẩn hóa điểm số

- Điểm số của một tài liệu d là tổng các điểm số nhân với số hệ thống thực sự trả về tài liệu đó

$$Score_{\text{CombMNZ}}(d) = Count(d) \times \sum_{i=1}^N score_i(d)$$

Count(d) là số lượng hệ thống trả về tài liệu d

- Mặc dù có những **vấn đề về chuẩn hóa** thường gặp trong các phương pháp dựa trên điểm số, CombMNZ vẫn cạnh tranh với các phương pháp dựa trên xếp hạng

Reciprocal Rank Fusion (RRF)

- RRF hoạt động bằng cách tổng hợp thứ hạng của các tài liệu từ nhiều nguồn khác nhau và đưa ra một danh sách xếp hạng kết hợp dựa trên công thức **reciprocal rank**
- RRF giả định nếu một tài liệu xuất hiện trong nhiều danh sách xếp hạng và có thứ hạng cao trong bất kỳ danh sách nào, thì tài liệu đó có khả năng liên quan cao hơn.
- **Hàm RRF** gán trọng số cho mỗi tài liệu theo nghịch đảo của vị trí của nó trong bảng xếp hạng
 - Ưu tiên các tài liệu ở "đầu" bảng xếp hạng
 - Phạt các tài liệu ở dưới "đầu" bảng xếp hạng

Reciprocal Rank Fusion (RRF)

- Giả sử có K danh sách xếp hạng từ các hệ thống hoặc mô hình khác nhau, với mỗi danh sách bao gồm các tài liệu $d_1, d_2, \dots, d_n$. Tài liệu d có thứ hạng $r_k(d)$ trong danh sách thứ k
- Tính điểm cho mỗi tài liệu:** Điểm của tài liệu d là tổng các nghịch đảo của thứ hạng mà nó nhận được trong mỗi danh sách

$$S(d) = \sum_{k=1}^K \frac{1}{r_k(d) + c}$$

- $r_k(d)$ là thứ hạng của tài liệu d trong danh sách thứ k .
- c là một hằng số nhỏ (thường được đặt là 60) nhằm tránh các tài liệu có thứ hạng quá cao (~ 1) chiếm ưu thế quá lớn.
- Xếp hạng tài liệu dựa trên điểm:** Tài liệu nào có điểm cao hơn sẽ được xếp hạng cao hơn trong danh sách.

Reciprocal Rank Fusion (RRF)

Ví dụ: Giả sử có hai mô hình M1 và M2 với các tài liệu được xếp hạng như sau:

- M1 xếp hạng: d1 (hạng 1), d2 (hạng 2), d3 (hạng 3).
- M2 xếp hạng: d2 (hạng 1), d3 (hạng 2), d1 (hạng 3).
- Với $c=60$, điểm số của mỗi tài liệu:

$$S(d_1) = \frac{1}{1+60} + \frac{1}{3+60} = \frac{1}{61} + \frac{1}{63}$$

$$S(d_2) = \frac{1}{2+60} + \frac{1}{1+60} = \frac{1}{62} + \frac{1}{61}$$

$$S(d_3) = \frac{1}{3+60} + \frac{1}{2+60} = \frac{1}{63} + \frac{1}{62}$$

- Xếp hạng lại dựa trên điểm số đã tính để có danh sách xếp hạng kết hợp.

Đặc điểm của RRF

- **Kết hợp thông tin từ nhiều nguồn:** tận dụng ưu điểm của mỗi mô hình.
- **Đơn giản nhưng hiệu quả:** Mặc dù RRF chỉ dựa vào thứ hạng (rank), nó thường đạt hiệu quả cao trong IR
- **Ít nhạy cảm với nhiễu:** Bằng cách sử dụng nghịch đảo thứ hạng, RRF ít bị ảnh hưởng bởi các tài liệu có thứ hạng kém trong một mô hình nhưng lại được xếp hạng cao trong các mô hình khác.

Ưu điểm:

1. Khả năng tổng hợp linh hoạt
2. Hiệu quả cao trên nhiều bài toán
3. Không yêu cầu điều chỉnh tham số phức tạp: Ngoại trừ hằng số c , RRF không đòi hỏi việc điều chỉnh quá nhiều tham số

Nhược điểm :

1. Phụ thuộc vào số lượng mô hình: phụ thuộc vào chất lượng và sự đa dạng của các mô hình được kết hợp
2. Thiên về thứ hạng hơn là điểm: có thể làm mất đi một số thông tin chi tiết mà các mô hình đã học được.

Unsupervised Reranking Methods

- ranx là một thư viện xếp hạng nhanh các chỉ số đánh giá trên Python, tận dụng Numba để thực hiện các phép toán vector tốc độ cao và tự động song song hóa

The link for ranx GitHub:

<https://github.com/AmenRa/ranx>



The link for the notebooks:

<https://github.com/AmenRa/ranx/tree/master/notebooks>

Installing ranx command:

```
! pip install ranx
```